LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 04

Singly Linked List



Disusun oleh:

Rayhan Ahza Widyamukti

109082500210

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

1. **Guided**

Source Code

```
#include <iostream>

using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L) {
    L.first = NULL;
}

address alokasi(infotype x) {
    address P = new elmlist;
    P->info = x;
    P->next = NULL;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = NULL;
}

void insertFirst(List &L, address P) {
    P->next = L.first;
    L.first = P;
}

void insertLast(List &L, address P) {
    if (L.first == NULL) {
        insertFirst(L, P);
    } else {
        address Q = L.first;
```

```
            while (Q->next != NULL) {
                Q = Q->next;
            }
            Q->next = P;
        }
    }

    void insertAfter(List &L, address P, address Prec) {
        if (Prec != NULL) {
            P->next = Prec->next;
            Prec->next = P;
        }
    }

    void deleteFirst(List &L, address &P) {
        if (L.first != NULL) {
            P = L.first;
            L.first = P->next;
            P->next = NULL;
        }
    }

    void deleteLast(List &L, address &P) {
        if (L.first != NULL) {
            if (L.first->next == NULL) {
                deleteFirst(L, P);
            } else {
                address Q = L.first;
                while (Q->next->next != NULL) {
                    Q = Q->next;
                }
                P = Q->next;
                Q->next = NULL;
            }
        }
    }

    void deleteAfter(List &L, address &P, address Prec) {
        if (Prec != NULL && Prec->next != NULL) {
            P = Prec->next;
            Prec->next = P->next;
            P->next = NULL;
        }
    }

    void printInfo(List L) {
        address P = L.first;
        while (P != NULL) {
```

```
            cout << P->info << ' ';
            P = P->next;
        }
        cout << '\n';
    }

    int main() {
        List L;
        address P1, P2, P3, P4, P5;

        createList(L);

        P1 = alokasi(2);
        insertFirst(L, P1);

        P2 = alokasi(0);
        insertFirst(L, P2);

        P3 = alokasi(8);
        insertFirst(L, P3);

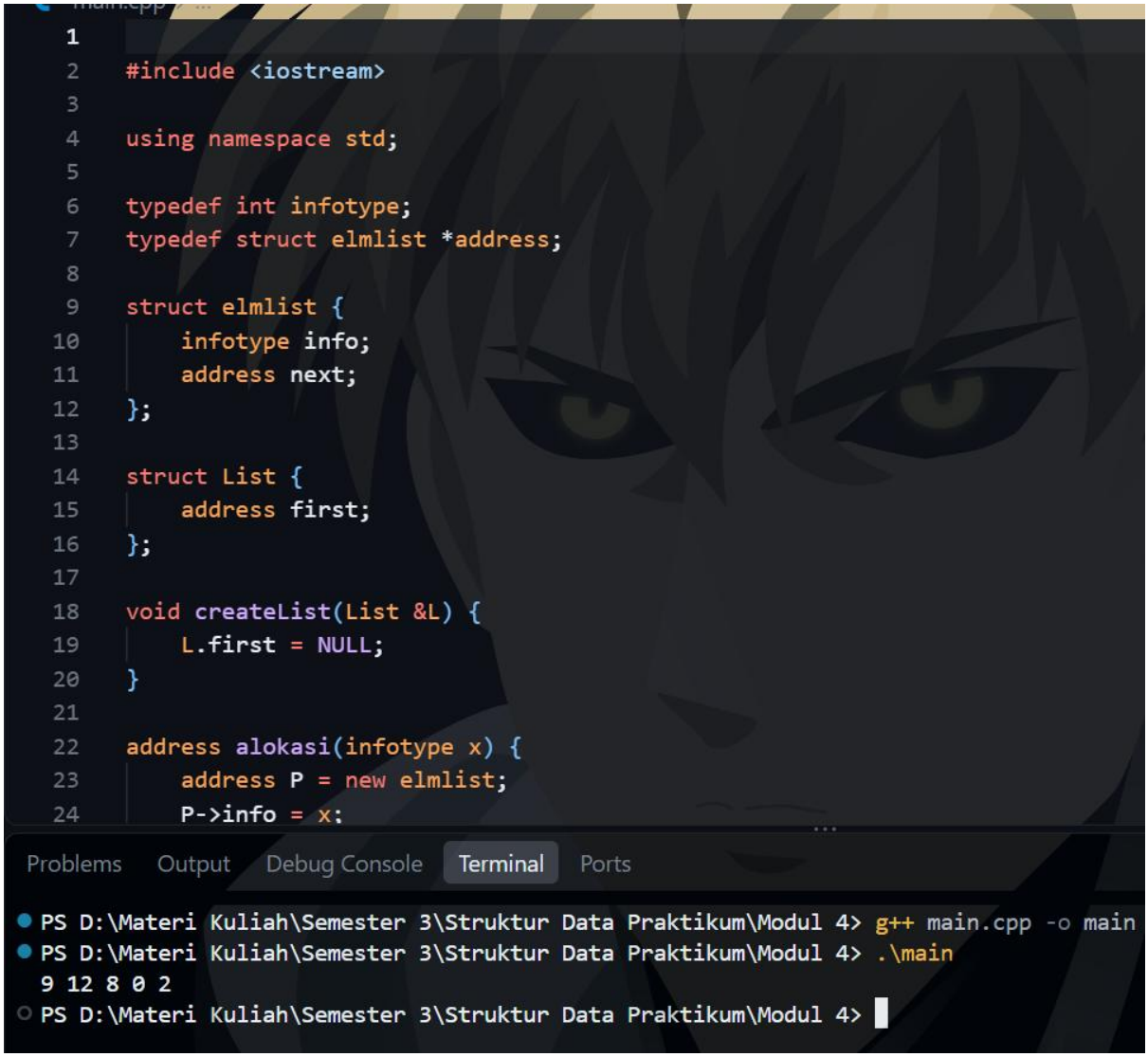
        P4 = alokasi(12);
        insertFirst(L, P4);

        P5 = alokasi(9);
        insertFirst(L, P5);

        printInfo(L);

        return 0;
    }
```

Screenshoot program



```cpp
#include <iostream>

using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L) {
    L.first = NULL;
}

address alokasi(infotype x) {
    address P = new elmlist;
    P->info = x;
```

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Materi Kuliah\Semester 3\Struktur Data Praktikum\Modul 4> g++ main.cpp -o main
PS D:\Materi Kuliah\Semester 3\Struktur Data Praktikum\Modul 4> .\main
9 12 8 0 2
PS D:\Materi Kuliah\Semester 3\Struktur Data Praktikum\Modul 4>
```

Deskripsi program

Penjelasan:

Program ini adalah implementasi Singly Linked List dengan tipe data int yang dibuat memakai struktur ADT (Abstract Data Type). Setiap node didefinisikan dengan struct elmlist yang berisi info untuk menyimpan data dan next untuk menunjuk ke node selanjutnya, sedangkan address adalah alias pointer ke elmlist. List-nya sendiri dibungkus dalam struct List yang hanya punya satu pointer bernama first sebagai penunjuk elemen pertama. Fungsi createList dipakai untuk membuat list kosong dengan mengisi first dengan NULL, lalu alokasi membuat node baru dan mengisi datanya, dan dealokasi menghapus node dari memori. Untuk operasi insert, ada insertFirst (menyisipkan di depan), insertLast (menyisipkan di belakang dengan menelusuri sampai node terakhir), dan insertAfter (menyisipkan setelah node tertentu). Untuk operasi delete, ada deleteFirst, deleteLast, dan deleteAfter, yang hanya melepas node dari list dan menyimpannya di parameter P supaya bisa dipakai lagi atau di-

dealokasi. Fungsi printInfo menelusuri list dari first sampai NULL sambil menampilkan isi tiap node. Di main, program membuat list kosong, lalu mengalokasikan lima node dengan data 2, 0, 8, 12, dan 9 yang semuanya dimasukkan memakai insertFirst. Karena insertFirst selalu menaruh node baru di depan, urutan datanya jadi terbalik dari urutan penambahan, sehingga keluaran programnya adalah 9 12 8 0 2.

2. **Soal 1**

Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Node
{
    string nama;
    Node *next;
};

Node *head = NULL;

void insertLast(string nama)
{
    Node *baru = new Node;
    baru->nama = nama;
    baru->next = NULL;

    if (head == NULL)
    {
        head = baru;
    }
    else
    {
        Node *bantu = head;
        while (bantu->next != NULL)
        {
            bantu = bantu->next;
        }
        bantu->next = baru;
    }
}

bool deleteFirst(string &namaDilayani)
{
    if (head == NULL)
```

```
    {
        return false;
    }
    Node *hapus = head;
    namaDilayani = hapus->nama;
    head = head->next;
    delete hapus;
    return true;
}

void tampilAntrean()
{
    cout << "Antrean: ";
    if (head == NULL)
    {
        cout << "(kosong)";
    }
    else
    {
        Node *bantu = head;
        while (bantu != NULL)
        {
            cout << bantu->nama;
            if (bantu->next != NULL)
            {
                cout << " -> ";
            }
            bantu = bantu->next;
        }
    }
    cout << endl;
}

int main()
{
    int pilihan;
    string nama;

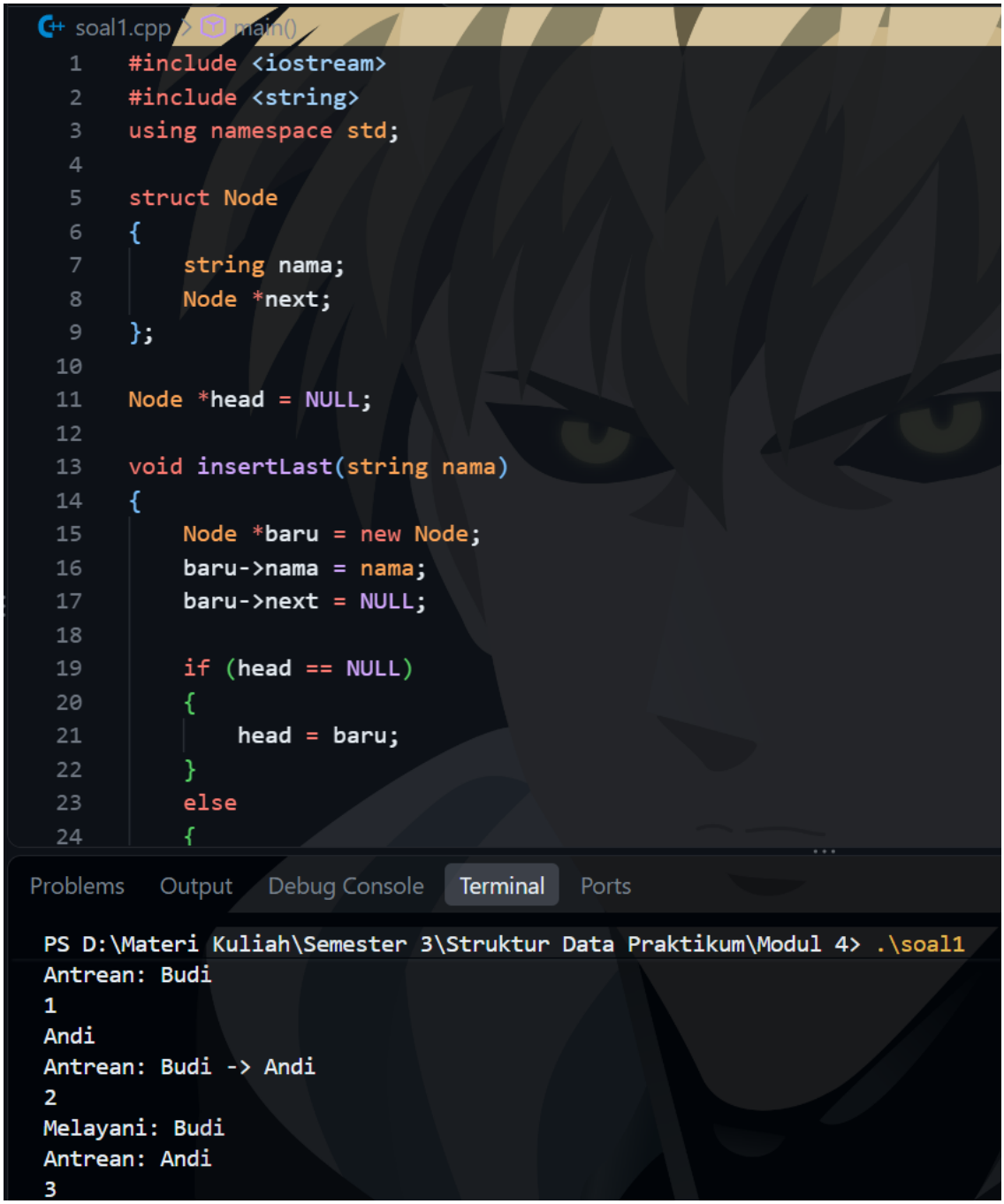
    do
    {
        cin >> pilihan;

        if (pilihan == 1)
        {
            cin >> nama;
            insertLast(nama);
            tampilAntrean();
        }
```

```
            else if (pilihan == 2)
            {
                if (deleteFirst(nama))
                {
                    cout << "Melayani: " << nama << endl;
                    tampilAntrean();
                }
                else
                {
                    cout << "Antrean kosong" << endl;
                }
            }
        } while (pilihan != 3);

        return 0;
    }
```

Screenshoot program



Deskripsi program

Program ini menggunakan Singly Linked List untuk menyimpan antrean nasabah bank, di mana setiap node berisi nama nasabah dan pointer next yang menunjuk ke node berikutnya. Pointer head dipakai sebagai penanda nasabah paling depan. Pada fungsi insertLast, node baru dibuat lalu disambungkan di akhir list dengan cara menelusuri node dari head sampai menemukan node yang next-nya NULL, sedangkan jika list masih kosong maka node baru langsung menjadi head. Pada fungsi deleteFirst, nama nasabah di head disimpan dulu, lalu head dipindah ke node selanjutnya dan node lama dihapus dengan delete supaya memori

tidak bocor. Antrean ini bersifat FIFO (First In First Out), jadi nasabah yang datang lebih dulu akan dilayani lebih dulu. Menu di main diulang menggunakan do-while sampai pengguna memilih angka 3 untuk keluar, dan setiap ada perubahan, fungsi tampilAntrean dipanggil untuk menampilkan kondisi antrean terbaru.

3. Soal 2

Source Code

```
#include <iostream>
using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist
{
    infotype info;
    address next;
};

struct List
{
    address first;
};

void createList(List &L)
{
    L.first = NULL;
}

address alokasi(infotype x)
{
    address P = new elmlist;
    P->info = x;
    P->next = NULL;
    return P;
}

void insertFirst(List &L, address P)
{
    P->next = L.first;
    L.first = P;
}

void insertLast(List &L, address P)
{
```

```
    if (L.first == NULL)
    {
        insertFirst(L, P);
    }
    else
    {
        address Q = L.first;
        while (Q->next != NULL)
        {
            Q = Q->next;
        }
        Q->next = P;
    }
}

void insertAfter(List &L, address P, address Prec)
{
    if (Prec != NULL)
    {
        P->next = Prec->next;
        Prec->next = P;
    }
}

address findElm(List L, infotype x)
{
    address P = L.first;
    while (P != NULL)
    {
        if (P->info == x)
        {
            return P;
        }
        P = P->next;
    }
    return NULL;
}

void printInfo(List L)
{
    address P = L.first;
    while (P != NULL)
    {
        cout << P->info;
        if (P->next != NULL)
        {
            cout << " -> ";
        }
```

```
            P = P->next;
        }
        cout << endl;
    }

    int main()
    {
        List L;
        int n, x, cari, baru;

        createList(L);

        cin >> n;
        for (int i = 0; i < n; i++)
        {
            cin >> x;
            insertLast(L, alokasi(x));
        }

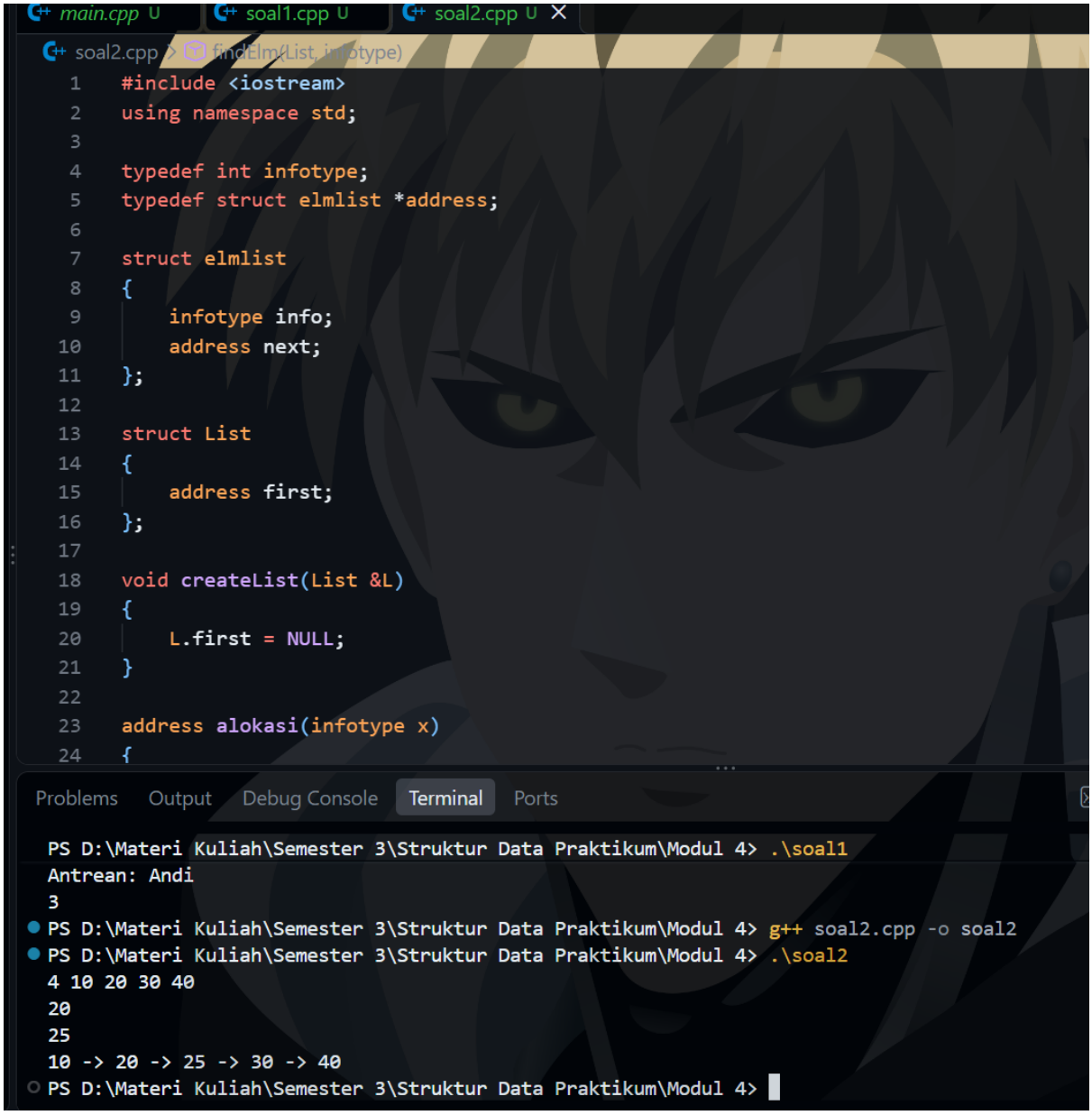
        cin >> cari;
        cin >> baru;

        address Prec = findElm(L, cari);
        if (Prec != NULL)
        {
            insertAfter(L, alokasi(baru), Prec);
        }

        printInfo(L);

        return 0;
    }
```

Screenshoot program



Deskripsi program

Program ini memakai Singly Linked List dengan struktur ADT, di mana setiap node (elmlist) berisi info untuk menyimpan bilangan dan next untuk menunjuk ke node berikutnya, sedangkan List hanya menyimpan pointer first sebagai penunjuk elemen pertama. Pertama, program membaca jumlah elemen n, lalu membaca n bilangan dan memasukkannya satu per satu dengan insertLast supaya urutan list sama dengan urutan masukan. Setelah itu program membaca bilangan yang dicari dan bilangan baru yang akan disisipkan. Fungsi findElm menelusuri list dari first sampai ketemu node yang info-nya sama dengan bilangan yang dicari, dan jika tidak ada maka mengembalikan NULL. Kalau node-nya ditemukan, node tersebut dijadikan Prec, lalu insertAfter menyisipkan node baru tepat setelahnya dengan cara menyambungkan next node baru ke next milik Prec, baru

kemudian next milik Prec diarahkan ke node baru. Urutan penyambungan ini penting supaya node setelahnya tidak terputus dari list. Terakhir, printInfo menampilkan seluruh isi list dengan pemisah ->. Jika data yang dicari tidak ada di list, penyisipan tidak dilakukan dan list ditampilkan apa adanya.

4. **Soal 3**

Source Code

```
#include <iostream>
using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist
{
    infotype info;
    address next;
};

struct List
{
    address first;
};

void createList(List &L)
{
    L.first = NULL;
}

address alokasi(infotype x)
{
    address P = new elmlist;
    P->info = x;
    P->next = NULL;
    return P;
}

void insertFirst(List &L, address P)
{
    P->next = L.first;
    L.first = P;
}

void insertLast(List &L, address P)
{
```

```
    if (L.first == NULL)
    {
        insertFirst(L, P);
    }
    else
    {
        address Q = L.first;
        while (Q->next != NULL)
        {
            Q = Q->next;
        }
        Q->next = P;
    }
}

void hitungSum(List L, int &total)
{
    total = 0;
    address P = L.first;
    while (P != NULL)
    {
        total = total + P->info;
        P = P->next;
    }
}

int main()
{
    List L;
    int n, x, total;

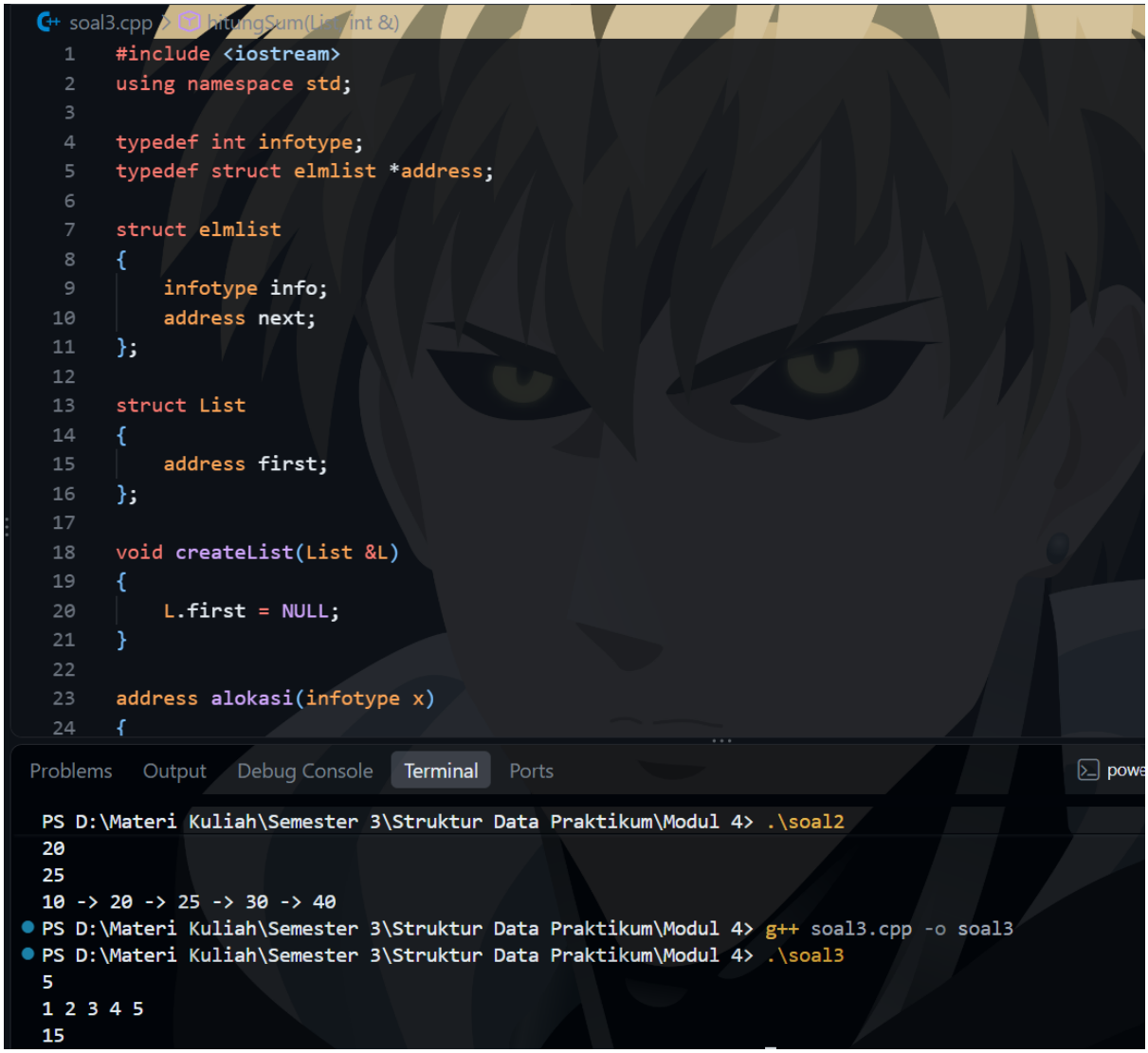
    createList(L);

    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cin >> x;
        insertLast(L, alokasi(x));
    }

    hitungSum(L, total);
    cout << total << endl;

    return 0;
}
```

Screenshoot program



```
soal3.cpp > hitungSum(List, int &)
1   #include <iostream>
2   using namespace std;
3
4   typedef int infotype;
5   typedef struct elmlist *address;
6
7   struct elmlist
8   {
9       infotype info;
10      address next;
11  };
12
13  struct List
14  {
15      address first;
16  };
17
18  void createList(List &L)
19  {
20      L.first = NULL;
21  }
22
23  address alokasi(infotype x)
24  {
```

```
Problems  Output  Debug Console  Terminal  Ports
PS D:\Materi Kuliah\Semester 3\Struktur Data Praktikum\Modul 4> .\soal2
20
25
10 -> 20 -> 25 -> 30 -> 40
PS D:\Materi Kuliah\Semester 3\Struktur Data Praktikum\Modul 4> g++ soal3.cpp -o soal3
PS D:\Materi Kuliah\Semester 3\Struktur Data Praktikum\Modul 4> .\soal3
5
1 2 3 4 5
15
```

Deskripsi program

Program ini memakai Singly Linked List dengan struktur ADT, di mana tiap node (elmlist) menyimpan info berupa bilangan bulat dan next yang menunjuk ke node berikutnya, sedangkan List hanya berisi pointer first sebagai penunjuk elemen pertama. Di main, program membaca jumlah elemen n, lalu membaca n bilangan dan memasukkannya satu per satu ke list dengan insertLast supaya urutannya sesuai masukan. Prosedur hitungSum bekerja dengan menyiapkan variabel total yang diawali 0, kemudian menelusuri list dari first memakai pointer P sampai P bernilai NULL, dan di setiap node nilai info ditambahkan ke total. Hasil penjumlahan dikirim keluar lewat parameter total yang bertipe referensi (int &total), lalu ditampilkan di main. Jadi untuk masukan 5 dan 1 2 3 4 5, keluarannya adalah 15.

5. **Soal 4**

Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Buku
{
    string judul;
    int tahun;
};

typedef Buku infotype;
typedef struct elmlist *address;

struct elmlist
{
    infotype info;
    address next;
};

struct List
{
    address first;
};

void createList(List &L)
{
    L.first = NULL;
}

address alokasi(infotype x)
{
    address P = new elmlist;
    P->info = x;
    P->next = NULL;
    return P;
}

void dealokasi(address &P)
{
    delete P;
    P = NULL;
}

void insertFirst(List &L, address P)
{
    P->next = L.first;
```

```
    L.first = P;
}

void insertLast(List &L, address P)
{
    if (L.first == NULL)
    {
        insertFirst(L, P);
    }
    else
    {
        address Q = L.first;
        while (Q->next != NULL)
        {
            Q = Q->next;
        }
        Q->next = P;
    }
}

void deleteFirst(List &L, address &P)
{
    if (L.first != NULL)
    {
        P = L.first;
        L.first = P->next;
        P->next = NULL;
    }
}

void deleteLast(List &L, address &P)
{
    if (L.first != NULL)
    {
        if (L.first->next == NULL)
        {
            deleteFirst(L, P);
        }
        else
        {
            address Q = L.first;
            while (Q->next->next != NULL)
            {
                Q = Q->next;
            }
            P = Q->next;
            Q->next = NULL;
        }
```

```
    }
}

void printInfo(List L)
{
    cout << "Daftar Buku:" << endl;
    address P = L.first;
    while (P != NULL)
    {
        cout << P->info.judul << " (" << P->info.tahun << ")"
<< endl;
        P = P->next;
    }
}

int main()
{
    List L;
    int n;

    createList(L);

    cout << "Jumlah buku: ";
    cin >> n;

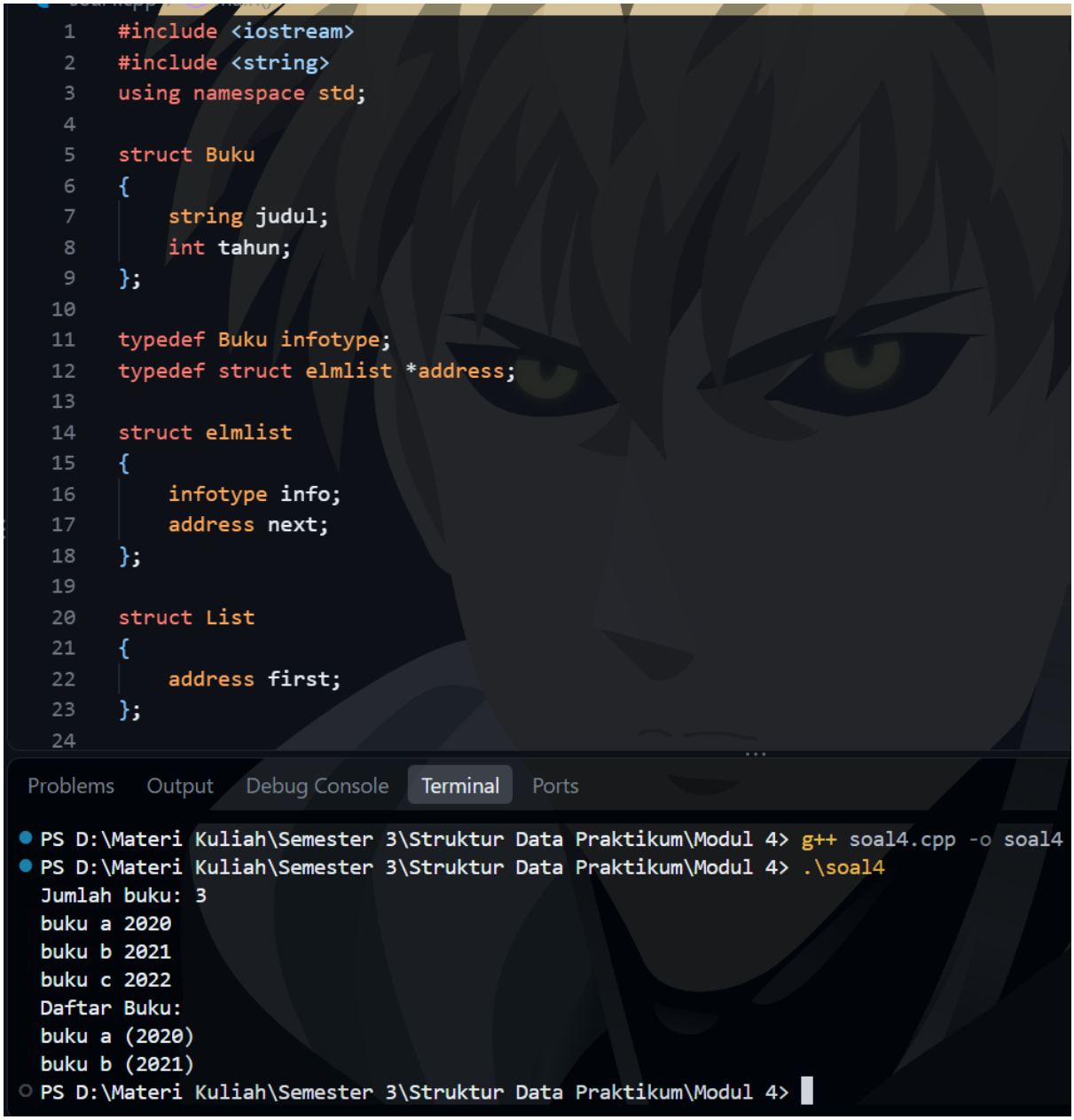
    for (int i = 0; i < n; i++)
    {
        Buku b;
        string kata1, kata2;
        cin >> kata1 >> kata2 >> b.tahun;
        b.judul = kata1 + " " + kata2;
        insertLast(L, alokasi(b));
    }

    address hapus = NULL;
    deleteLast(L, hapus);
    if (hapus != NULL)
    {
        dealokasi(hapus);
    }

    printInfo(L);

    return 0;
}
```

Screenshoot program



```cpp
#include <iostream>
#include <string>
using namespace std;

struct Buku
{
    string judul;
    int tahun;
};

typedef Buku infotype;
typedef struct elmlist *address;

struct elmlist
{
    infotype info;
    address next;
};

struct List
{
    address first;
};
```

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Materi Kuliah\Semester 3\Struktur Data Praktikum\Modul 4> g++ soal4.cpp -o soal4
PS D:\Materi Kuliah\Semester 3\Struktur Data Praktikum\Modul 4> .\soal4
Jumlah buku: 3
buku a 2020
buku b 2021
buku c 2022
Daftar Buku:
buku a (2020)
buku b (2021)
PS D:\Materi Kuliah\Semester 3\Struktur Data Praktikum\Modul 4>
```

Deskripsi program

Program ini memakai Singly Linked List untuk menyimpan data buku, di mana isi tiap node (info) berupa struct Buku yang berisi judul bertipe string dan tahun bertipe int. Di main, program pertama-tama meminta jumlah buku n, lalu perulangan for membaca tepat n data buku sehingga input tidak tak terbatas. Karena judul pada contoh terdiri dari dua kata (misalnya “Buku A”), judul dibaca sebagai kata1 dan kata2 lalu digabung dengan spasi, sedangkan tahun dibaca langsung ke b.tahun. Setiap buku dimasukkan ke list memakai insertLast supaya urutannya sesuai masukan. Setelah semua data masuk, deleteLast dipanggil satu kali: jika list hanya punya satu elemen maka cukup memanggil deleteFirst,

sedangkan jika lebih dari satu, pointer Q berjalan sampai Q->next->next bernilai NULL (satu node sebelum yang terakhir), lalu node terakhir disimpan di P dan Q->next diisi NULL supaya terlepas dari list. Node yang sudah lepas dihapus dari memori dengan dealokasi, kemudian printInfo menampilkan sisa buku dengan format Judul (Tahun).